Полиморфизм

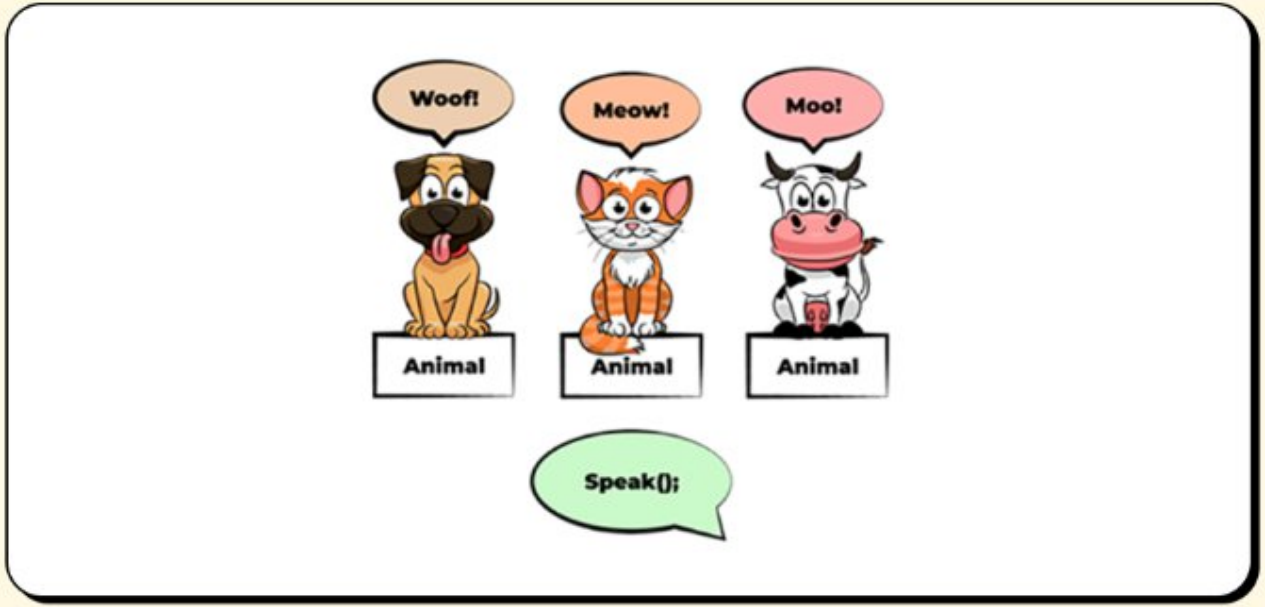
Александра Волосова,
к.т.н., доцент кафедры
ИУ5

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

Часть 1

Определение. Классификации

Polymorphism



poly — много и **morph** — форма, вид

Классическое определение (Гради Буч*):

Положение теории типов, согласно которому имена (например, переменных) могут обозначать объекты разных (но имеющих общего родителя) классов. Следовательно, любой объект, обозначаемый полиморфным именем, может по-своему реагировать на некий общий набор операций

Полиморфизм — использование производных классов через указатели или ссылки на базовый класс

* Гради Буч – автор методов ООанализа и ООпроектирования

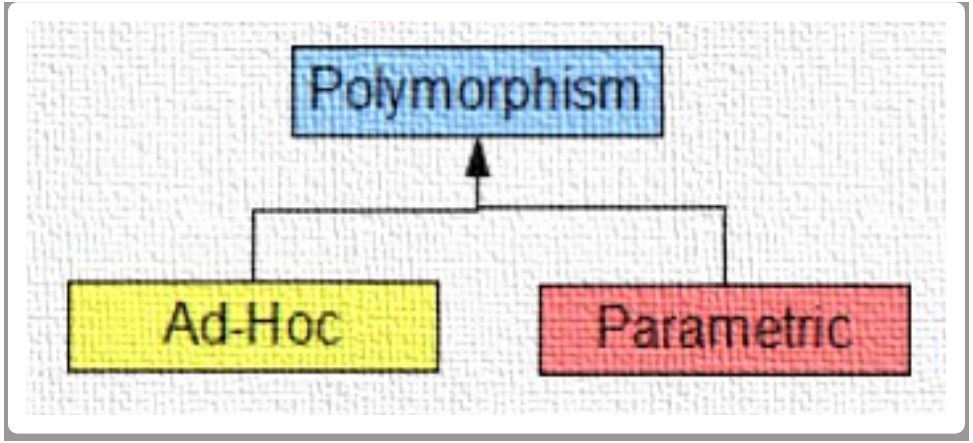
- Полиморфизм — **способность объекта** выполнять **специализированные действия** на основе его типа
- Полиморфизм — **свойство системы** использовать **объекты с одинаковым интерфейсом** без информации о типе и внутренней структуре объекта.
- Полиморфизм — **способность объекта** использовать **методы производного класса**, который не существует на момент создания базового.
- Полиморфизм — **свойство**, которое позволяет одно и тоже имя использовать **для решения** нескольких **технически разных задач**.
- Полиморфизм — **один интерфейс**, **множество методов**
- Полиморфизм — **возможность** применения **одноименных методов** с одинаковыми или различными наборами параметров в одном классе или в группе классов, связанных отношением наследования.
- Полиморфизм — это **свойство** программных сущностей **работать сходным** образом с **данными разных типов**

Классификации полиморфизма

Кристофер Стрейчи (1967) неформально ввел понятие полиморфизма в процедурные языки программирования, выделив функции, в которых:

- Работа с разными типами аргументов отличается
- Работа с разными типами аргументов унифицирована

Он определил первое как **специальный полиморфизм**, а второе – как **параметрический полиморфизм**



Классификации полиморфизма

Карделли и Вегнер (1985) включили в классификацию Стрейчи объектно-ориентированные языки.

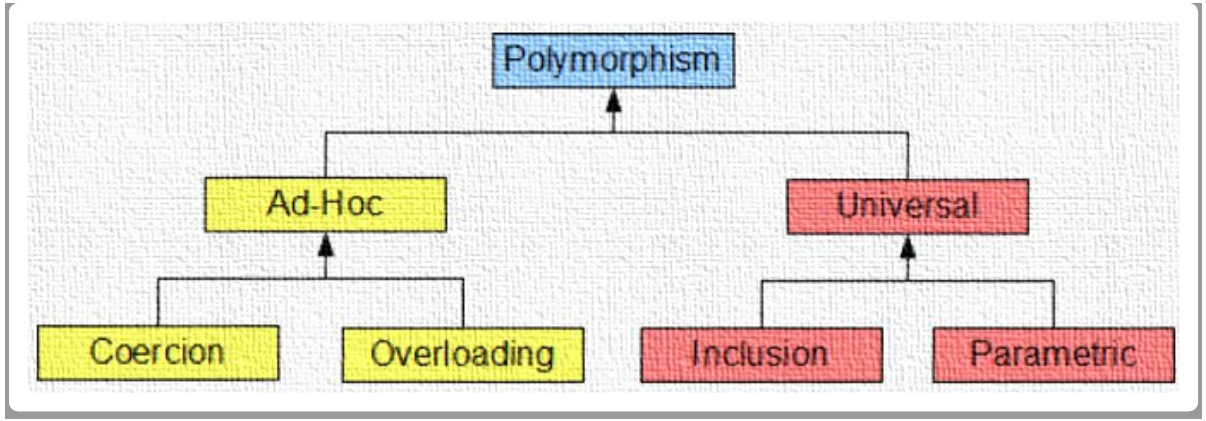
Они выделили функции, которые:

1. Работают с **конечным набором** различных и потенциально **не связанных между собой типов**

- **Coercion**
- **Overloading**

2. Работают с потенциально **бесконечным количеством типов** в рамках единой структуры

- **Inclusion** (только ООП)
- **Parametric**



Ad-hoc (специальный) полиморфизм

– специальный полиморфизм, который также называется перегрузочным (перегрузка функций и операторов в ООЯ) специальным полиморфизмом

Позволяет функциям с одинаковыми именами работать по-разному для разных типов

C++

```
#include <iostream>
using namespace std;
int sum(int x, int y)
{  int c = x + y;
   return c;
}
string sum(const char* x, const char* y)
{
   string summation(x);
   summation += y;
   return summation; }
```

```
int main()
{
   cout << sum(50, 20)
      << " :- Integer addition Output\n";
   cout << sum("Polymorphism", "
achieved")
      << " :- String Concatenation Output\n";
}
```

Time:	Memory:
0.0000 secs	3.46 Mb

Your Output

```
70 :- Integer addition Output
Polymorphism achieved :- String Concatenation Output
```

Ad-hoc (специальный) полиморфизм

Java

```
import java.util.*;

public class Main {
    public static int sum(int x, int y)
    {
        int c = x + y;
        return c;
    }

    public static String sum(String x, String y)
    {
        String summation = x.concat(y);
        return summation;
    }

    public static void main(String[] args)
    {
        System.out.println(sum(50, 20)
                    + " :- Integer addition Output");
        System.out.println(
            sum("Polymorphism", " achieved")
            + " :- String Concatenation Output");
    }
}
```

Time:	Memory:
0.0200 secs	38.028 Mb

Your Output

```
55
Early
```

Ad-hoc (специальный) полиморфизм

C#

```
using System;
class Program
{
    static int Sum(int x, int y)
    {
        int c = x + y;
        return c;
    }
static string SumStr(string x, string y)
    {
        string summation = x + y;
        return summation;
    }
 // Main function
    static void Main(string[] args)
    {
        Console.WriteLine($"{Sum(50, 20)} :- Integer
addition Output");
        Console.WriteLine($"{SumStr("Polymorphism", "
achieved")} :- String Concatenation Output");
    }
}
```

Time:	Memory:
0.0200 secs	14.812 Mb

Your Output

```
70 :- Integer addition Output
Polymorphism achieved :- String Concatenation Output
```

Ad-hoc (специальный) полиморфизм

Python3

```
def sum(x, y):
    c = x + y
    return c

def sum_str(x, y):
    summation = x + y
    return summation

# Main function
if __name__ == '__main__':
    print(f"{sum(50, 20)} :- Integer addition Output")
    print(f"{sum_str('Polymorphism', ' achieved')} :-
String Concatenation Output")
```

Time:	Memory:
0.0100 secs	8.96 Mb

Your Output

```
70 :- Integer addition Output
Polymorphism achieved :- String Concatenation Output
```

Ad-hoc (специальный) полиморфизм

Kotlin

```
fun sum(x: Int, y: Int) = x + y
fun sumStr(x: String, y: String) = x + y


fun main() {
    println("${sum(50, 20)} :- Integer addition Output")
    println("${sumStr("Polymorphism", " achieved")} :- String
Concatenation Output")
}
```

Time:	Memory:
0.1500 secs	49.448 Mb

Your Output

```
70 :- Integer addition Output
Polymorphism achieved :- String Concatenation Output
```

Coercion (принудительный) полиморфизм

Принудительный полиморфизм. Принудительный полиморфизм возникает, когда объект или примитив приводится к какому-либо другому типу. Приведение может быть:

1. **Неявным.** Неявное приведение осуществляется самим компилятором. Например: float f=100 (целое число неявно преобразуется в число с плавающей запятой).
2. **Явным.** При явном приведении типов используются такие выражения, как **const_cast, dynamic_cast** и т. д. Например: если в классе определен оператор преобразования для какого-либо типа, например «int», то его можно использовать в любом месте программы, где ожидается работа с данными целочисленного типа.

Coercion (принудительный) полиморфизм

```
class IntClass {
   private int num;

   public IntClass(int a) {
      num = a;
   }

   public int intValue() {
      return num;
   } // conversion from User-defined type to Basic type
}

class Main {
   public static void show(int x) {
      System.out.println(x);
   }

   public static void main(String[] args) {
      IntClass i = new IntClass(100);
      show(746);
      show(i.intValue());
   }
}
```

Java

Time:	Memory:
0.0300 secs	37.888 Mb

Your Output

```
746
100
```

Coercion (принудительный) полиморфизм

```
#include <iostream>
using namespace std;

class IntClass {
   int num;

public:
   IntClass(int a)
      : num(a)
   {
   }

   operator int() const
   {
      return num;
   } // conversion from User-defined type to Basic type
};

void show(int x)
{
   cout << x << endl;
}

int main()
{
   IntClass i = 100;
   show(746); // outputs 746
   show(i); // outputs 100
}
```

C++

Time:	Memory:
0.0300 secs	37.888 Mb

Your Output

```
746
100
```

Coercion (принудительный) полиморфизм

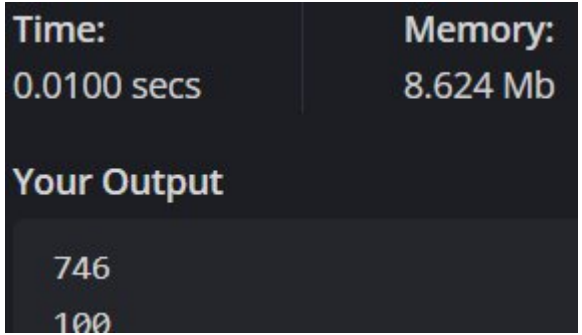
```
class IntClass:
    def __init__(self, a):
        self.num = a

    def intValue(self):
        return self.num
    # conversion from User-defined type to Basic type

def show(x):
    print(x)

i = IntClass(100)
show(746)
show(i.intValue())
```

Python



Coercion (принудительный) полиморфизм

```
using System;

class IntClass {
   private int num;

   public IntClass(int a) {
      num = a;
   }

   public static implicit operator int(IntClass i) {
      return i.num;
   } // conversion from User-defined type to Basic type
}

class MainClass {
   public static void show(int x) {
      Console.WriteLine(x);
   }
   public static void Main(string[] args) {
      IntClass i = new IntClass(100);
      show(746);
      show(i);
   }
}
```

C#

Time:	Memory:
0.0200 secs	14.76 Mb

Your Output

```
746
100
```

Coercion (принудительный) полиморфизм

```
class IntClass(private val num: Int) {
    fun toInt(): Int = num
}
// Extension function to convert IntClass to Int
operator fun IntClass.invoke(): Int = toInt()

fun show(x: Int) {
    println(x)
}

fun main() {
    val i = IntClass(100)
    show(746)

    // Method 1: Explicit conversion
    show(i.toInt())
}
```

Kotlin

Time:	Memory:
0.1800 secs	49.584 Mb

Your Output

```
746
100
```

Coercion (принудительный) полиморфизм

позволяет преобразовывать тип аргумента в соответствии с типом соответствующего параметра функции.

- Если компилятор обнаруживает несоответствие между типом аргумента при вызове функции и типом соответствующего параметра, язык допускает преобразование типа аргумента в тип соответствующего параметра.
- Компилятор вставляет код, необходимый для выполнения преобразования.
- Само определение функции выполняется только для одного типа — типа ее параметра.

Принуждение может осуществляться двумя способами:

Сужение типа аргумента (сужающее приведение)

Расширение типа аргумента (расширяющее приведение)

```
One argument (10)
One argument (12)
One argument (65)
```

```
#include <iostream>
void display(int a) {
   std::cout << "One argument (" << a << ')';
}
int main( ) {
   display(10);
   std::cout << std::endl;
   display(12.6); // narrowing
   std::cout << std::endl;
   display('A'); // promotion
   std::cout << std::endl; }
```

- Приведение типов устраняет ошибки, связанные с несоответствием типов или отсутствием определения функции
- В C++ приведение типов выполняется во время компиляции. Если компилятор обнаруживает несоответствие типов, которое можно привести, он вставляет код преобразования непосредственно перед вызовом функции
- Большинство языков программирования поддерживают приведение типов. Например, в языке C типы аргументов при вызове функций сужаются и расширяются, так что одна и та же функция может принимать аргументы ограниченного набора типов.

Overloading (перегрузка)

- Перегрузка позволяет использовать различные варианты сигнатуры функции
- Перегрузка позволяет связывать вызовы функций с одним и тем же идентификатором, но разными типами аргументов, с определениями функций, имеющими соответствующие типы параметров.
- Одно и то же имя функции может использоваться с различными типами аргументов, не связанными между собой. Каждый набор типов аргументов имеет собственное определение функции. Компилятор связывает вызов функции с соответствующим определением функции.
- **В отличие от приведения типов**, перегрузка не предполагает наличия общей логики в определениях функций с одинаковыми идентификаторами. Каждое определение работает только со своим набором типов. Количество перегруженных функций ограничено количеством определений, реализованных в исходном коде.

```
#include <iostream>
void display() {
   std::cout << "No arguments";
}
void display(int a) {
   std::cout << "One argument (" << a << ')';
}
int main( ) {
   display();
   std::cout << std::endl;
   display(10);
   std::cout << std::endl; }
```

```
No arguments
One argument (10)
```

Перегрузка

- Перегрузка устраняет ошибки, связанные с множественными определениями функций
- В C++ перегрузка реализуется на этапе компиляции путем переименования каждой функции с идентичным именем в функцию с собственным уникальным идентификатором: язык искажает исходный идентификатор, добавляя к нему типы параметров, чтобы сгенерировать уникальное имя
- Компоновщик использует искаженное имя для привязки вызова функции к соответствующему определению
- Процедурный язык (C) не допускает перегрузки и требует уникального имени для каждого определения функции

Inclusion (включение)

- **Полиморфизм включения — это возможность использовать производные классы через указатели и ссылки на базовые классы.** Это полиморфизм во время выполнения, поскольку адрес функции не определяется компилятором во время компиляции, а для вызова функции во время выполнения разыменовывается правильный указатель из виртуальной таблицы. Для реализации inclusion полиморфизма используется концепция виртуальной функции, также известная как динамическая компоновка. Использование виртуальной функции позволяет выбирать вызываемую функцию в зависимости от типа объекта, для которого она вызывается

Inclusion (включение)

```
#include <iostream>
using namespace std;
class Image {
public:
   Image()
   {   }
   virtual void display() = 0;
};
class Jpg : public Image {
public:
   Jpg()
   {   }
   void display()
   {
      cout << "JPG Image File" << endl;
   }
};
class Png : public Image {
public:
   Png()
   {   }
   void display()
   {      cout << "PNG Image File" << endl;
   }
};
```

```
int main()
{
   Image* img;
   Jpg jg;
   Png pg;

   // stores the address of Jpg
   img = &jg;

   // invoking display() func of Jpg
   img->display();

   // stores the address of Png
   img = &pg;

   // invoking display() func of Png
   img->display();

   return 0;
}
```

В коде - два разных класса с функцией с одинаковым именем, но с разной реализацией

C++

Time:	Memory:
0.0000 secs	3.484 Mb

Your Output

```
JPG Image File
PNG Image File
```

Inclusion (включение)

```
abstract class Image {
    public Image() {
    }
    abstract void display();
}

class Jpg extends Image {
    public Jpg() {
    }

    @Override
    void display() {
        System.out.println("JPG Image File");
    }
}

class Png extends Image {
    public Png() {
    }

    @Override
    void display() {
        System.out.println("PNG Image File");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Image img;
        Jpg jg = new Jpg();
        Png pg = new Png();

        // stores the reference of Jpg
        img = jg;

        // invoking display() method of Jpg
        img.display();

        // stores the reference of Png
        img = pg;

        // invoking display() method of Png
        img.display();
    }
}
```

Time:	Memory:
0.0200 secs	37.972 Mb

Your Output

```
JPG Image File
PNG Image File
```

Java

Inclusion (включение)

```
abstract class Image {
    public Image() {
    }
    abstract void display();
}

class Jpg extends Image {
    public Jpg() {
    }

    @Override
    void display() {
        System.out.println("JPG Image File");
    }
}

class Png extends Image {
    public Png() {
    }

    @Override
    void display() {
        System.out.println("PNG Image File");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Image img;
        Jpg jg = new Jpg();
        Png pg = new Png();

        // stores the reference of Jpg
        img = jg;

        // invoking display() method of Jpg
        img.display();

        // stores the reference of Png
        img = pg;

        // invoking display() method of Png
        img.display();
    }
}
```

Time:	Memory:
0.0100 secs	8.788 Mb

Your Output

```
JPG Image File
PNG Image File
```

Python

Inclusion (включение)

```
using System;
abstract class Image
{
   public Image()
   {   }
public abstract void Display();
}
class Jpg : Image
{
   public Jpg()
   {   }
   public override void Display()
   {
      Console.WriteLine("JPG Image File");
   }
}
class Png : Image
{
   public Png()
   {   }
   public override void Display()
   {
      Console.WriteLine("PNG Image File");
   }
}
```

```
class MainClass
{
   public static void Main(string[] args)
   {
      Image img;
      Jpg jg = new Jpg();
      Png pg = new Png();

      // stores the reference of Jpg
      img = jg;

      // invoking display() method of Jpg
      img.Display();

      // stores the reference of Png
      img = pg;

      // invoking display() method of Png
      img.Display();
   }
}
```

Time:	Memory:
0.0300 secs	14.704 Mb

Your Output

```
JPG Image File
PNG Image File
```

C#

Inclusion (включение)

```
import kotlin.io.println

abstract class Image {
    abstract fun display()
}

class Jpg : Image() {
    override fun display() {
        println("JPG Image File")
    }
}

class Png : Image() {
    override fun display() {
        println("PNG Image File")
    }
}
```

```
fun main(args: Array<String>) {
    var img: Image
    val jg = Jpg()
    val pg = Png()

    // stores the reference of Jpg
    img = jg

    // invoking display() method of Jpg
    img.display()

    // stores the reference of Png
    img = pg

    // invoking display() method of Png
    img.display()
}
```

Time:	Memory:
0.1800 secs	49.692 Mb

Your Output

```
JPG Image File
PNG Image File
```

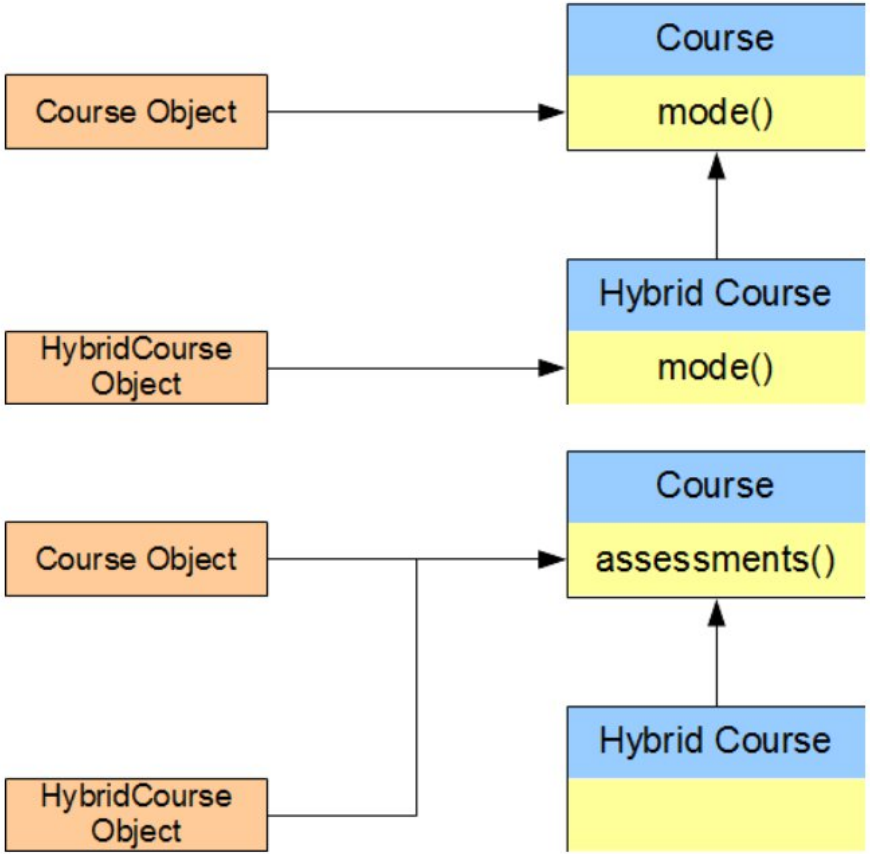
Kotlin

Inclusion (включение)

- Полиморфизм inclusion: в коде одно имя функции, но в зависимости от того, для какого объекта она вызывается, выполняется разный код.
- Полиморфизм inclusion допускает использование нескольких определений функций-членов, выбирая определение из набора в зависимости от типа объекта. Все определения функций-членов имеют одно и то же имя во всей иерархии.

Inclusion (включение)

- **HybridCourse** и **Course** принадлежат к одной иерархии.
- **HybridCourse** использует один способ доставки, а **Course** — другой.
- **mode()** запрос к объекту **Course** возвращает результат, отличный от **mode()** запроса к о объекту **HybridCourse**



В рамках полиморфизма inclusion классы связаны отношением is-a

Inclusion (включение)

```
#include <iostream>
#include "Course.h"
using std::cout;
using std::endl;

int main( ) {
   Course abc123("Intro to OO")
   HybridCourse abc124("Intro to OO*");
   cout << abc123.assessments() << endl;
   cout << abc123.mode() << endl;
   cout << abc124.assessments() << endl;
   cout << abc124.mode() << endl;
}
```

- Операции, идентичные для всех типов в иерархии, требуют определения только одной функции-члена.
- Запрос assessments() к HybridCourse объекту вызывает ту же функцию, что и запрос к Course объекту
- Определение запроса для HybridCourse класса приведет лишь к дублированию существующего кода и не является необходимым.

Полиморфизм inclusion устраняет дублирование логики в иерархии, не приводя к ошибкам из-за отсутствия определения функции.

В C++ полиморфизм включения реализуется во время выполнения с помощью виртуальных таблиц.

Parametric (параметрический)

- Параметрический (или обобщённый) полиморфизм устраняет различия между типами аргументов при вызове функции и типами параметров в определении функции
- Параметрический полиморфизм позволяет использовать в определениях функций идентичную логику независимо от типа. В отличие от приведения типов, логика является общей для всех возможных типов без каких-либо ограничений.
- Параметрический полиморфизм устраняет дублирование логики во всех типах, не вызывая ошибку отсутствия определения функции.
- В C++ параметрический полиморфизм реализуется во время компиляции с помощью синтаксиса шаблонов.

Дженерик (Generic)

- механизм параметрического полиморфизма, позволяющий писать код, работающий с объектами разного типа. При этом на этапе компиляции сохраняетя строгая типизация
- Дженерики используются в языках C#, Java, Kotlin, Rust и др.
- C++ использует шаблоны, более мощный механизм
- Python использует утиную типизация

Parametric (параметрический)

Типы не обязательно должны быть связаны между собой.

Например, функция, сортирующая int, использует ту же логику, что и функция, сортирующая Students. Если функция для сортировки int уже написана, нужно убедиться, что в классе Student есть оператор сравнения, идентичный тому, который используется в функции sorts

C++

```
#include <iostream>
template<typename T>
void sort(T* a, int n) {
   int i, j;
   T temp;
   for (i = n - 1; i > 0; i--) {
      for (j = 0; j < i; j++) {
         if (a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
         } } } }
class Student {
   int no;
   // other data - omitted here
public:
   Student(int n = 0) : no(n) {}
   bool operator>(const Student& rhs) const {
      return no > rhs.no;  }
   void display(std::ostream& os) const {os << no << std::endl;  } };
```

```
int main( ) {
   int m[] = {189, 843, 321};
   Student s[] = {Student(1256), Student(1267), Student(1234)};

   sort(m, 3);
   for (int i = 0; i < 3; i++)
      std::cout << m[i] << std::endl;

   sort(s, 3);
   for (int i = 0; i < 3; i++)
      s[i].display(std::cout);
}
```

```
189
321
843
1234
1256
1267
```

Parametric (параметрический)

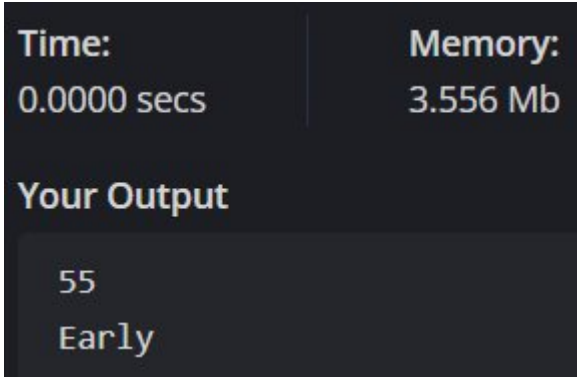
C++

```
#include <iostream>
#include <string>
using namespace std;

template <class temp>
temp greater(temp a, temp b)
{ if (a > b)
      return a;
   else
      return b;
}
int main()
{
   cout << ::greater(55, 11) << endl;

   string str1("Early"), str2("Binding");
   cout << ::greater(str1, str2) << endl;
}
```

Параметрический полиморфизм называется ранним связыванием и реализуется с помощью шаблонов



Parametric (параметрический)

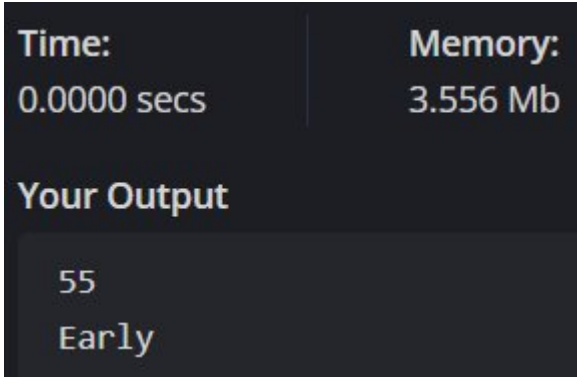
C++

```
#include <iostream>
#include <string>
using namespace std;

template <class temp>
temp greater(temp a, temp b)
{ if (a > b)
      return a;
   else
      return b;
}
int main()
{
   cout << ::greater(55, 11) << endl;

   string str1("Early"), str2("Binding");
   cout << ::greater(str1, str2) << endl;
}
```

Параметрический полиморфизм называется ранним связыванием и реализуется с помощью шаблонов



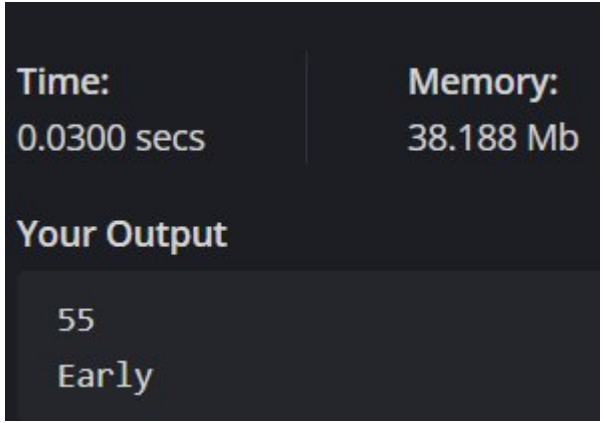
Parametric (параметрический)

```
public class Main {
   // Generic method to find the greater of two values
   public static <T extends Comparable<T>> T greater(T a, T
b) {
      if (a.compareTo(b) > 0) {
         return a;
      } else {
         return b;
      }
   }

   public static void main(String[] args) {
      // Example with integers
      System.out.println(greater(55, 11));

      // Example with strings
      String str1 = "Early";
      String str2 = "Binding";
      System.out.println(greater(str1, str2));
   }
}
```

Java



Parametric (параметрический)

```
def greater(a, b):
    """
    Function to return the greater of two values.

    Parameters:
    - a: First value
    - b: Second value

    Returns:
    - The greater of the two values
    """
    if a > b:  # Check if a is greater than b
        return a  # Return a if it's greater
    else:
        return b  # Otherwise, return b
if __name__ == "__main__":
    # Test with integer values
    print(greater(55, 11))
    # Test with string values
    str1 = "Early"
    str2 = "Binding"
    print(greater(str1, str2))
```

Python

Time:	Memory:
0.0100 secs	8.828 Mb

Your Output

```
55
Early
```

Parametric (параметрический)

```
/**
 * Function to return the greater of two values.
 *
 * @param a First value
 * @param b Second value
 * @return The greater of the two values
 */
fun <T : Comparable<T>> greater(a: T, b: T): T {
    return if (a > b) a else b
}

fun main() {
    // Test with integer values
    println(greater(55, 11))
    // Test with string values
    val str1 = "Early"
    val str2 = "Binding"
    println(greater(str1, str2))
}
```

Kotlin

Time:	Memory:
0.2500 secs	49.604 Mb

Your Output

```
55
Early
```

Parametric (параметрический)

```
using System;
public class GreaterDemo {
   // Generic method to find the greater of two values
   public static T Greater<T>(T a, T b) where T :
IComparable<T> {
      if (a.CompareTo(b) > 0) {
         return a;
      } else {
         return b;
      }
   }
   public static void Main(string[] args) {
      // Example with integers
      Console.WriteLine(Greater(55, 11));

      // Example with strings
      string str1 = "Early";
      string str2 = "Binding";
      Console.WriteLine(Greater(str1, str2));
   }
}
```

C#

Time:	Memory:
0.0400 secs	15.476 Mb

Your Output

```
55
Early
```

Тип	Характеристика	Реализация
Inclusion	Один интерфейс — разное поведение	Во время работы программы
Overload (перегрузка)	Одно имя — разные варианты	Во время компиляции
Coercion (приведение)	Тип преобразуется в процессе компиляции	Во время компиляции
Parametric (параметрический)	Один шаблон— любые типы	Во время компиляции